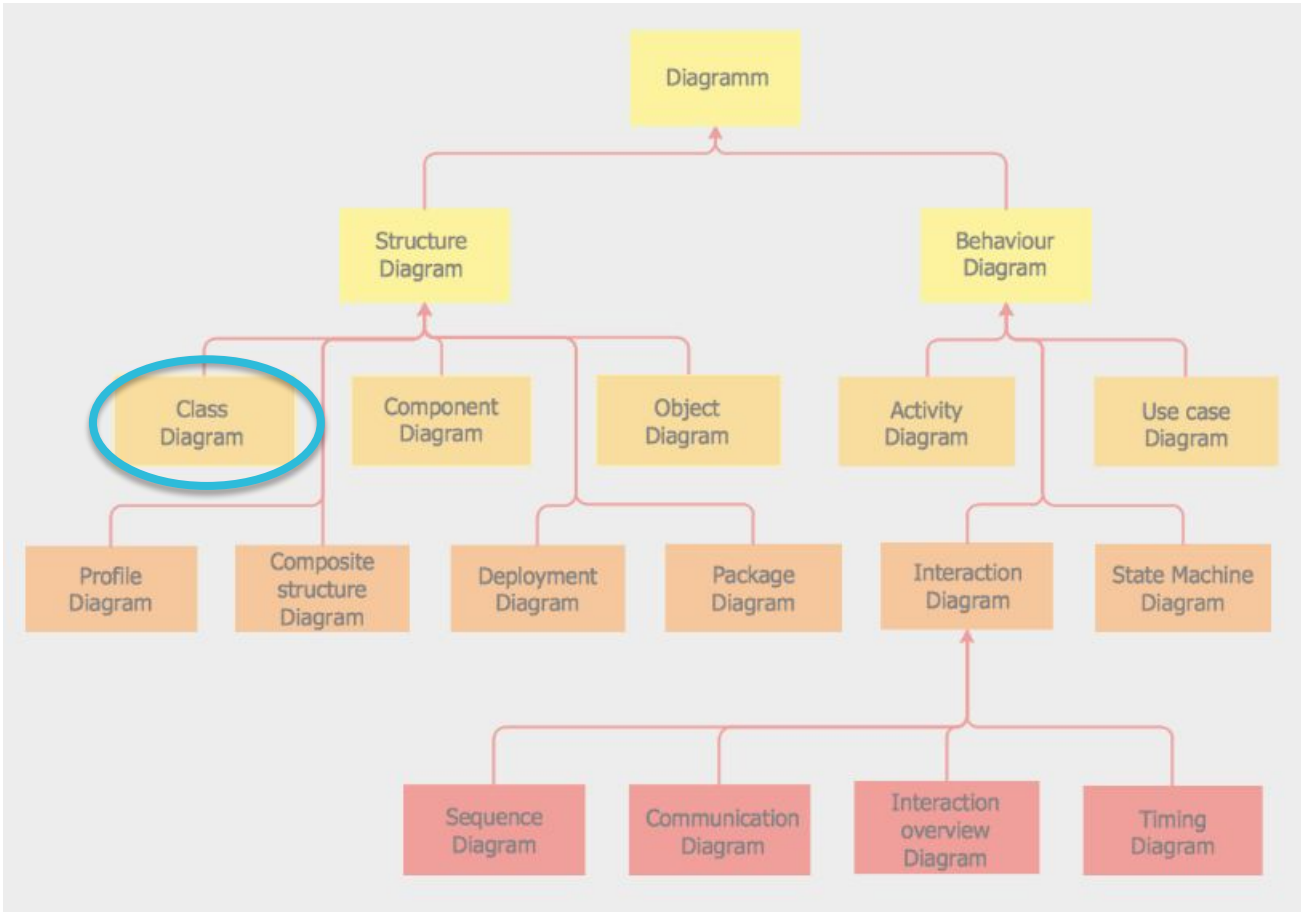


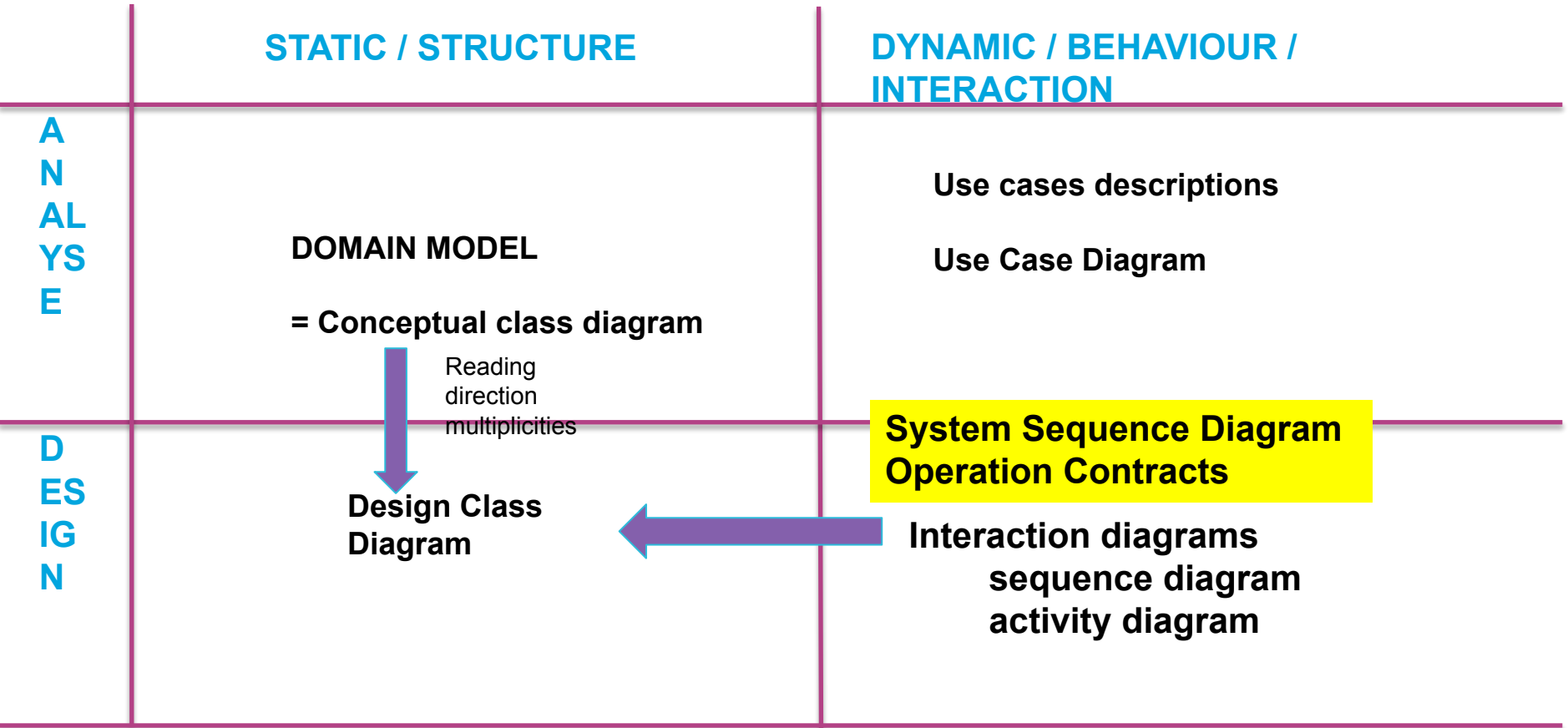
Design Class Diagram

Design Class Diagram (DCD)

Context



Conceptual Thinking 1: Analysis -> Design



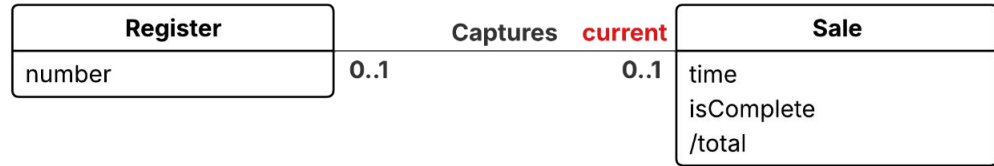
Conceptual (CCD) ⇔ Design Class Diagram (DCD)

Domain model

What are the differences?

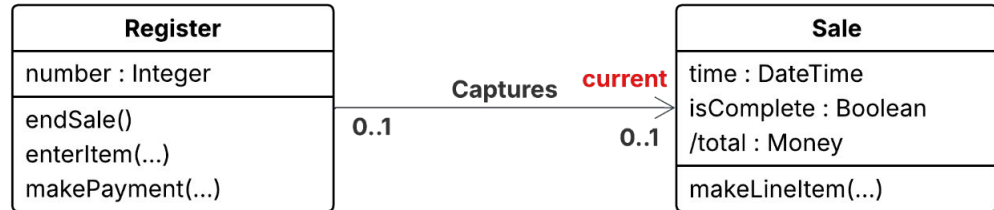
Conceptual Class Diagram
Domain Model

conceptual /ERD perspective



Design Class Diagram

software perspective



Conceptual (CCD) ⇔ Design Class Diagram (DCD)

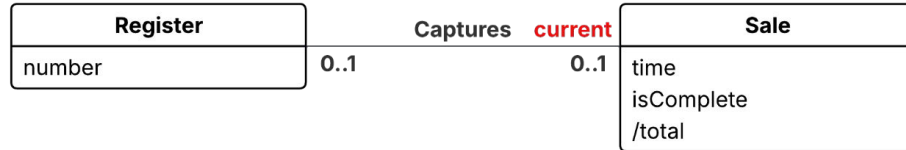
Domain model

From domain model to design class diagram by adding/adapting:

1. Attribute types and attribute accessibility
2. Design associations
3. Methods (with parameters and their types, return types, accessibility)
4. Design classes (see GRASP)
5. Other added information to a DCD

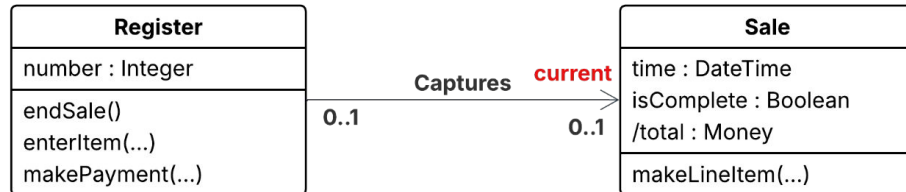
Conceptual Class Diagram
Domain Model

conceptual /ERD perspective



Design Class Diagram

software perspective

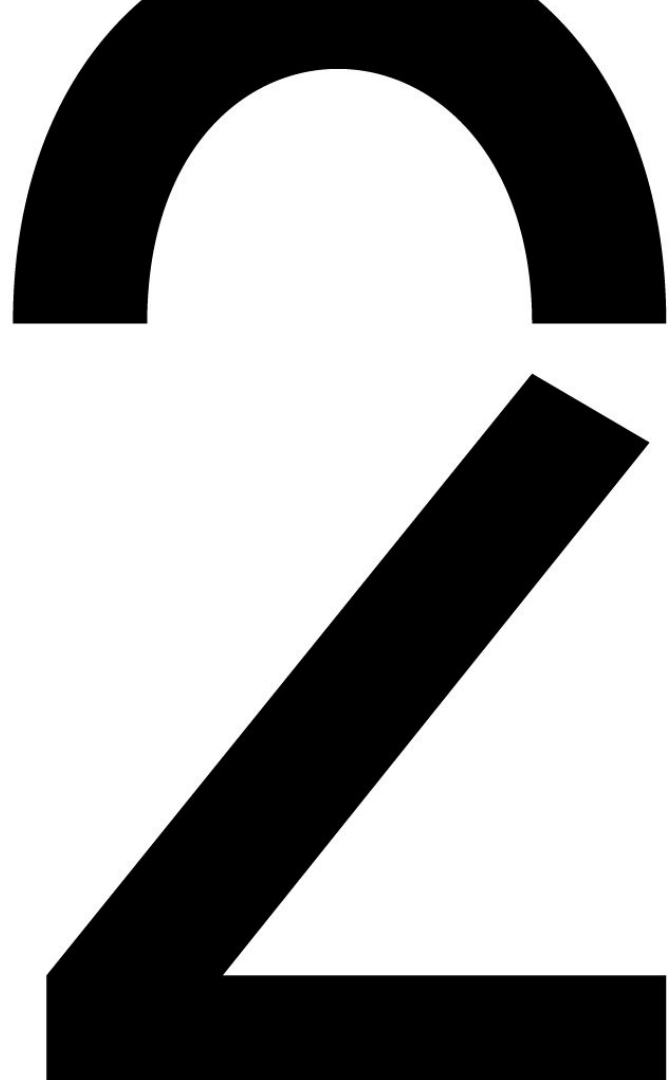


Conceptual (CCD) ⇔ Design Class Diagram (DCD)

	Conceptual Class Diagram Domain model	Design Class Diagram
Stage	Analysis	Design
Perspective	Conceptual / ERD	Software / Programming
Classes	Database objects	Software objects
Associaties	All relevant relationships between classes No visibility = no arrows	Enhanced with technical details Arrows indicate visibility
Methods	No	Operations with parameters and their data types, return data types, accessibility
Attributes	Concept properties	+ extra (private member vars) datatypes and accessibility

→ a well-designed design class diagram is an **enriched version** of the domain model

**DCD from CCD+SD
from example**



The Princess and the Frog

We'll start with a SD (last week's lesson content).

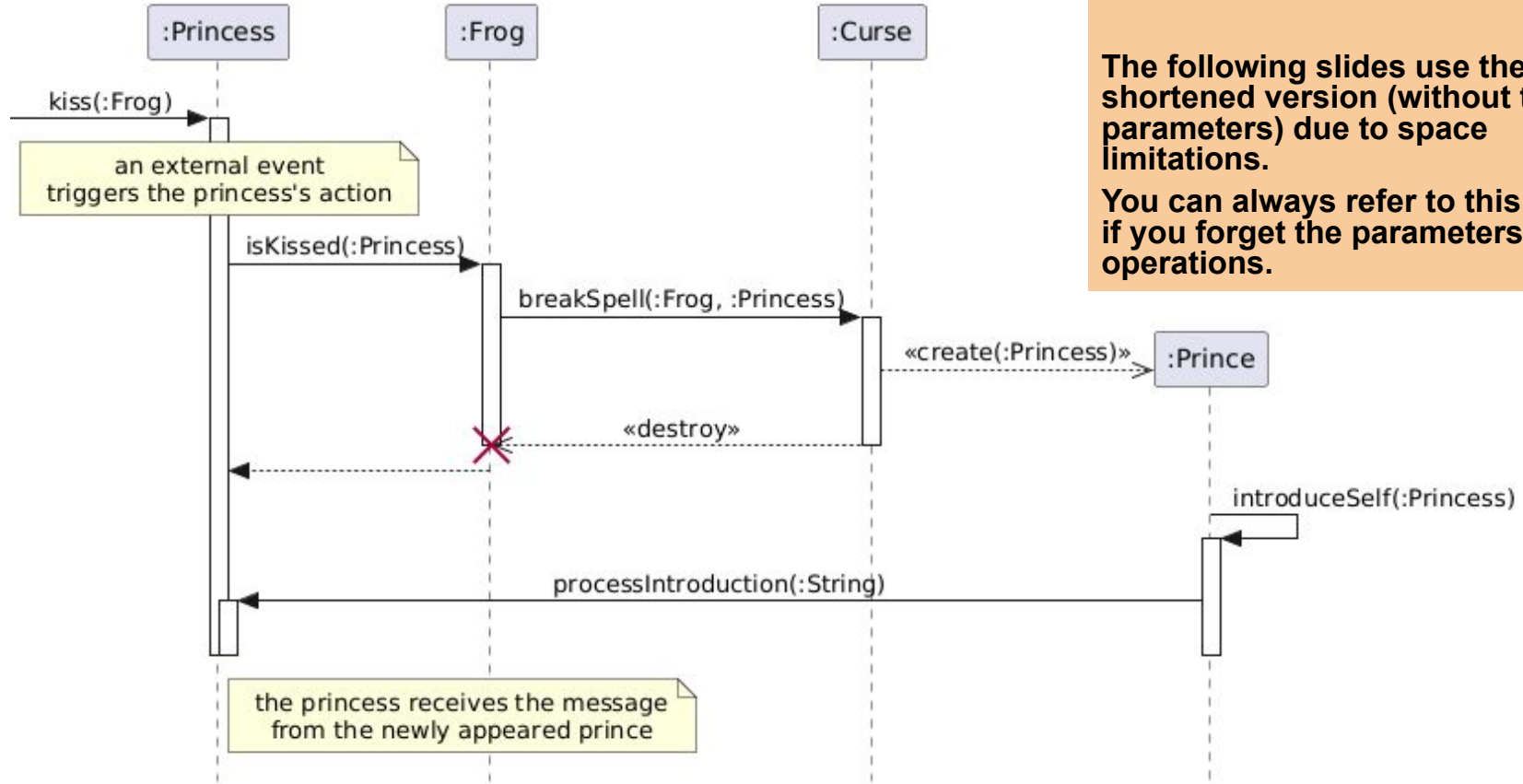
In this case, a passage from "The Princess and the Frog"

The moment the princess kisses the frog is modeled.

The original SD with all the operation parameters is shown only on the next slide.
From then on, due to space limitations, a shortened version is used.

In this section, we will go step by step, to build the design class diagram (DCD) using the example from last week.

Sequence Diagram - The Frog Prince

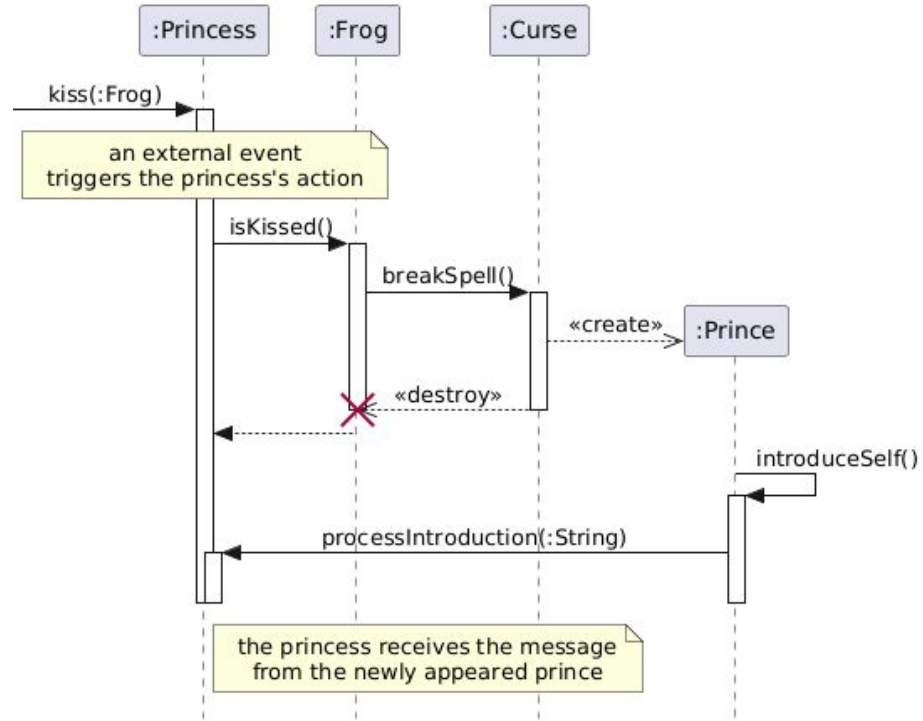


SD with all parameters

The following slides use the shortened version (without the parameters) due to space limitations.

You can always refer to this version if you forget the parameters of the operations.

Sequence Diagram - The Frog Prince



'Smaller' version where the parameters of each operation are not shown (but they are still there)

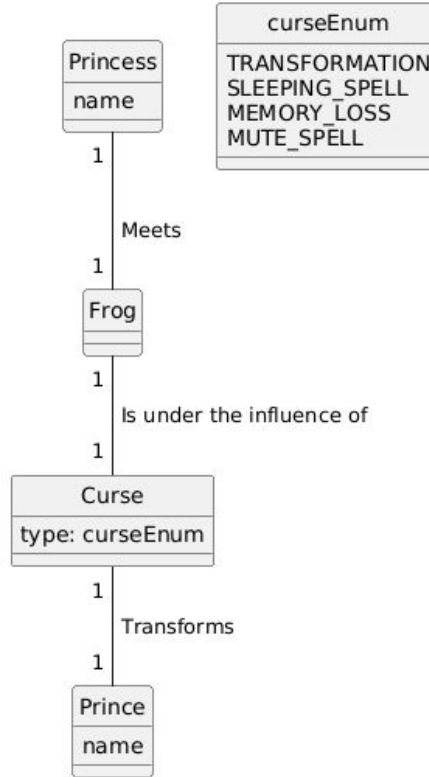
The Frog Prince – Domain model

Following the storyline, we obtain a domain model like the one in the left diagram.

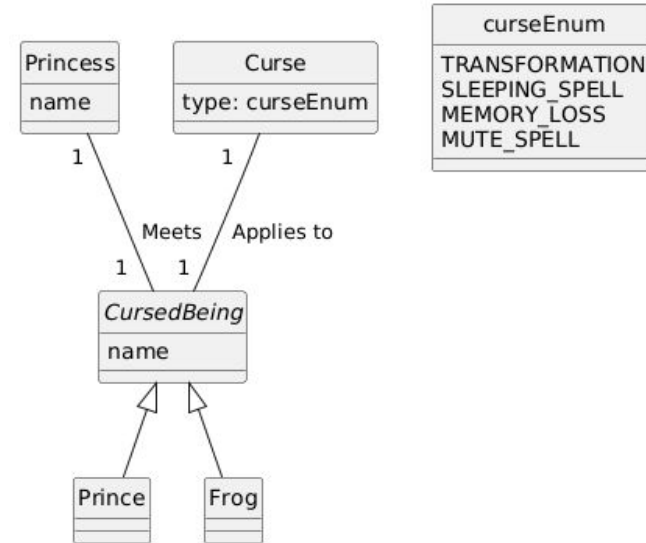
Since the prince and the frog are essentially the same "being", we could also generalize this to an abstract class, *CursedBeing*, as shown in the domain model in the right diagram.

We'll continue with the left diagram.

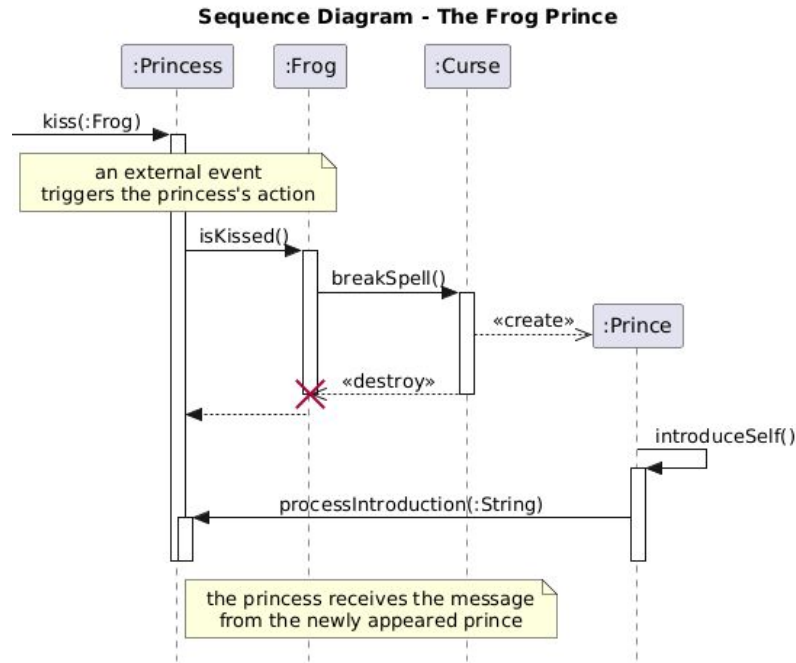
Domain model - The Frog Prince



Domain model - The Frog Prince



Frog Prince – Design Class Diagram (DCD)



Design Classes

We don't seem to detect any design classes in our SD; all classes are already present in the domain model.

However, a 'hidden' design class is hiding behind the **found message**.

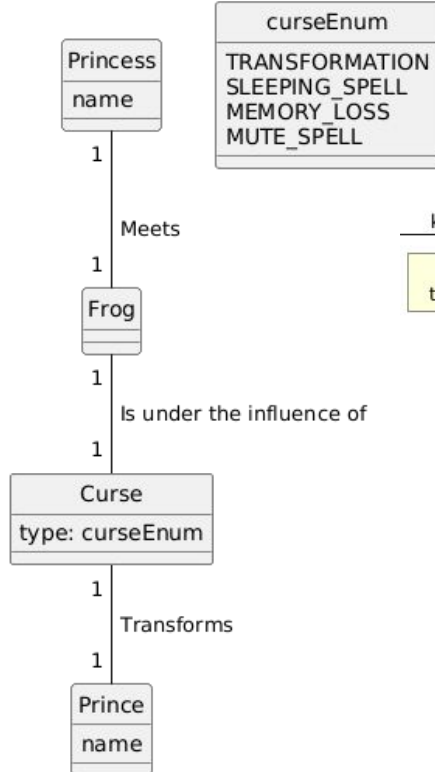
Who or what triggers the princess?

We solve this by adding a 'controller' at the application level = something that will cause that external trigger, allowing the princess to perform the operation behind the **found message** to carry out.

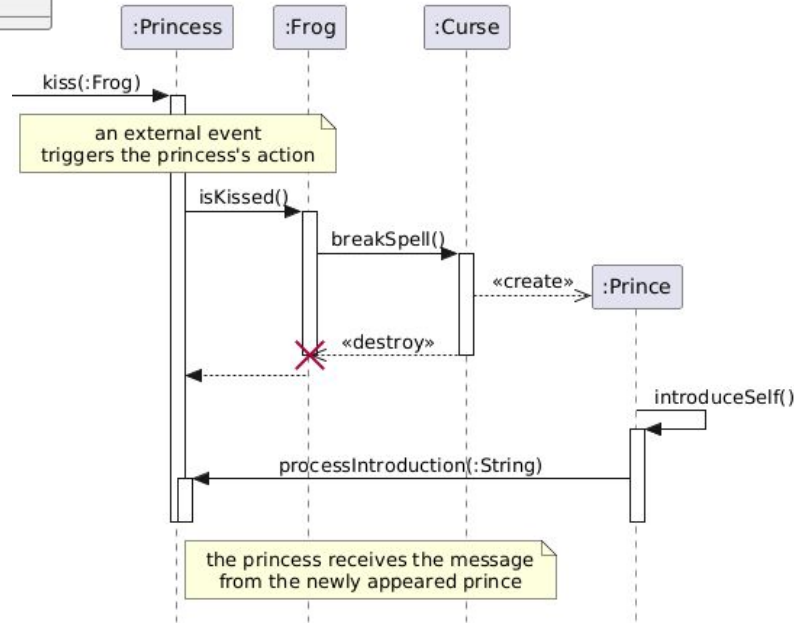
In Java, this will later be the class containing the **main function** to start the application. Keep in mind that this class will also need to create objects to begin execution (in this case, the princess and the frog to perform the first operation).

Frog Prince – Design Class Diagram (DCD)

Domain model - The Frog Prince



Sequence Diagram - The Frog Prince



If you have created multiple SDs for a single domain model (for example, because you have one SD per OC), you must follow the steps below **for all SDs**.

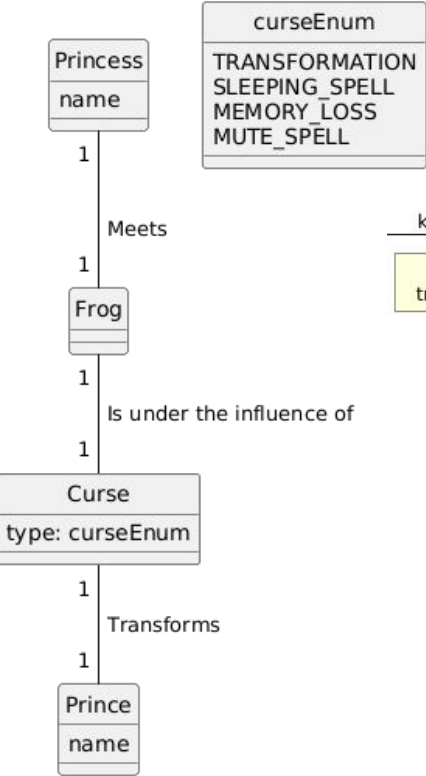
In our example, the single SD corresponds to the entire domain model.

We will now derive the DCD from the domain model and SD using the following **step-by-step plan**:

1. Start from the domain model and add the data types
2. Add the design classes (don't forget the app controller)
3. Add the operations, taking into account the parameters and return objects
4. Add the missing objects (which are needed to perform the operations) as an attribute of the calling class
5. Was `<<create>>` executed somewhere? Then draw the association with `<<create>>` (don't forget the app controller)
6. Finally, draw all dependencies with a dashed line and an arrow indicating their direction.
Who uses whom?
For now, we will draw this as `<<uses>>`.

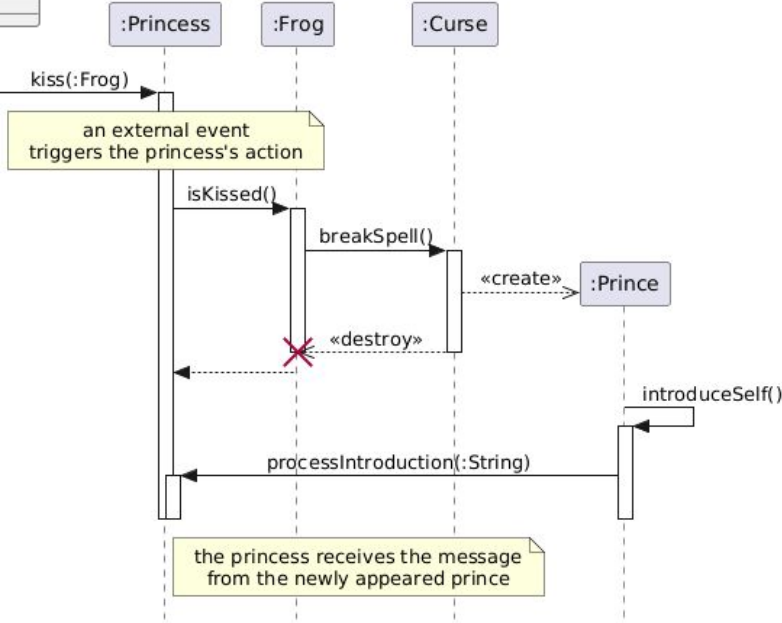
Frog Prince – DCD - Step 1

Domain model - The Frog Prince

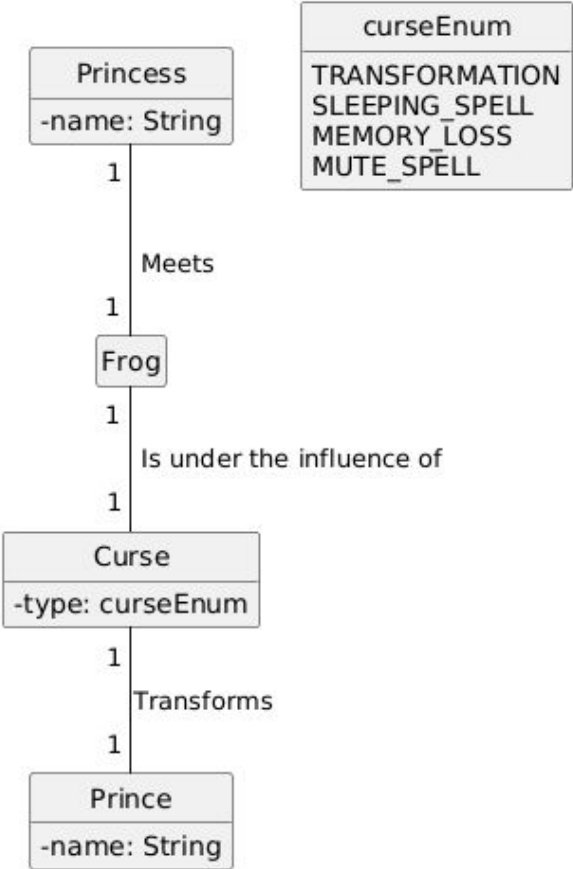


1. Start from the domain model and add the data types

Sequence Diagram - The Frog Prince

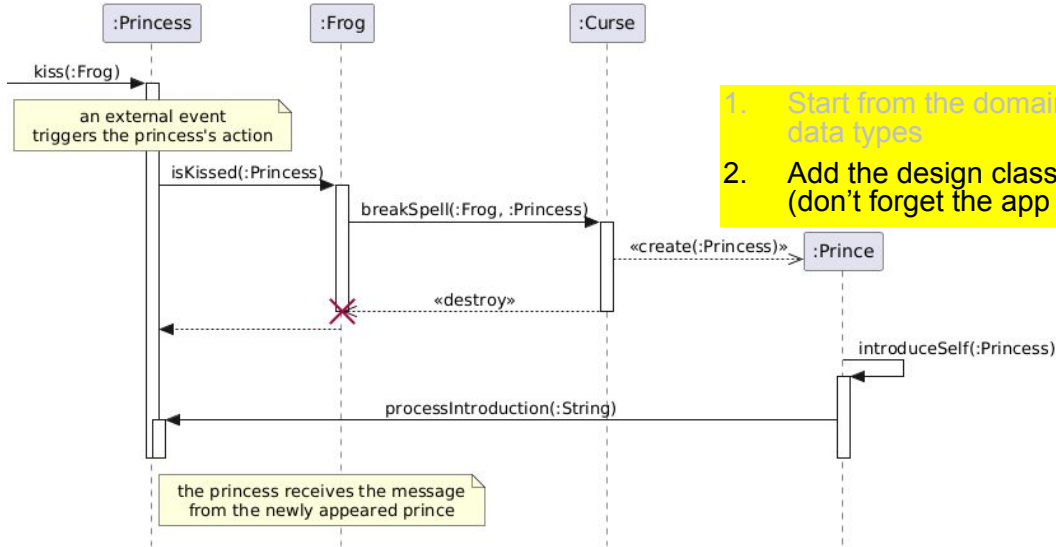


DCD - The Frog Prince



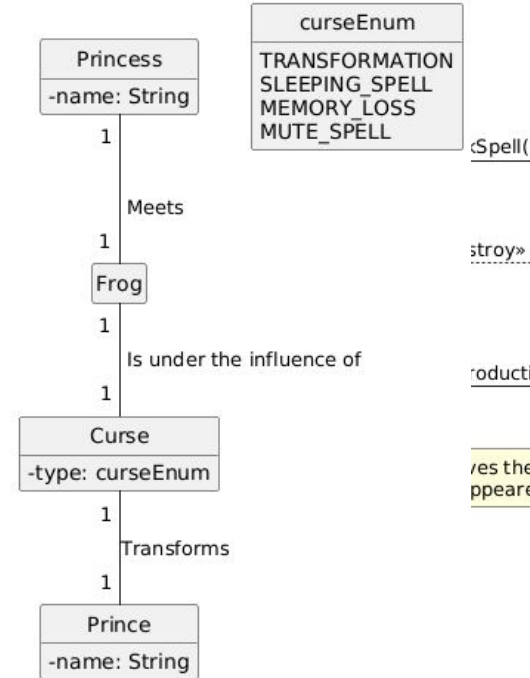
Frog Prince – DCD - Step 2

Sequence Diagram - The Frog Prince



1. Start from the domain model and add the data types
2. Add the design classes (don't forget the app controller)

DCD - The Frog Prince

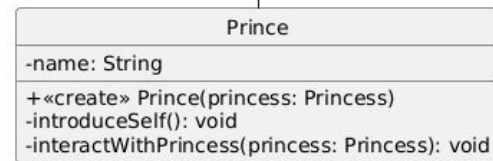
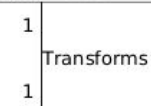
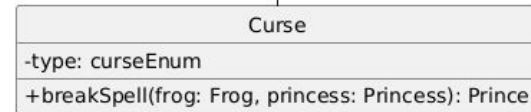
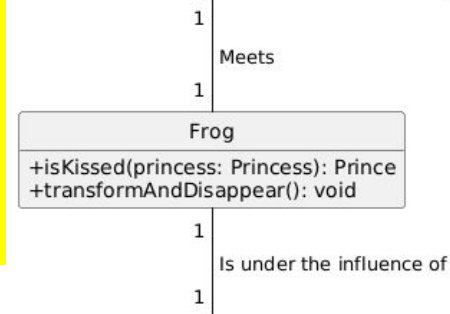
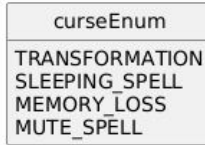
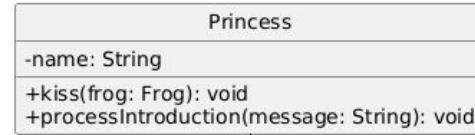


Frog Prince – DCD - Step 3

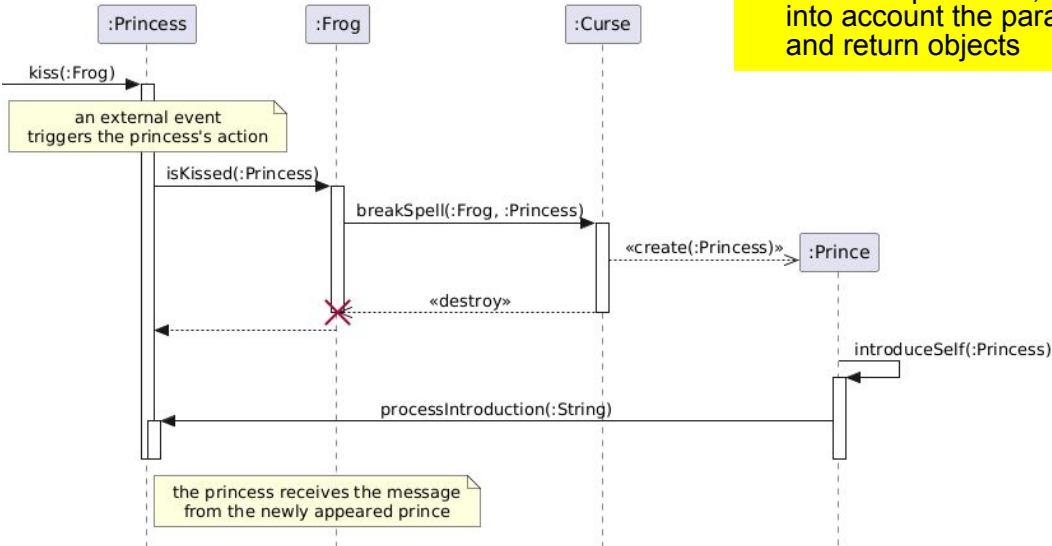


1. Start from the domain model and add the data types
2. Add the design classes (don't forget the app controller)
3. Add the operations, taking into account the parameters and return objects

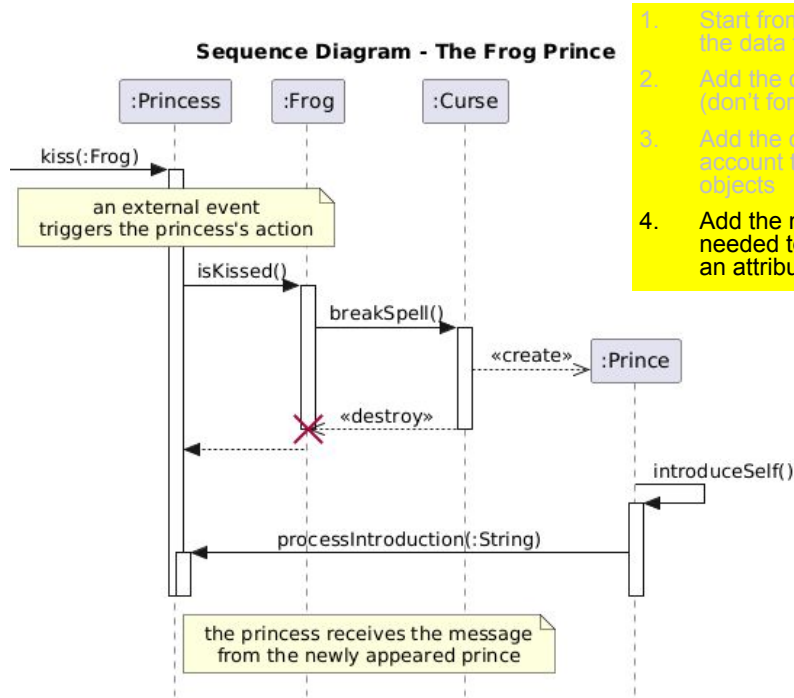
DCD - The Frog Prince



Sequence Diagram - The Frog Prince

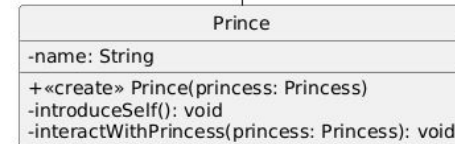
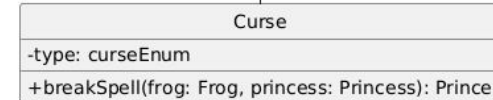
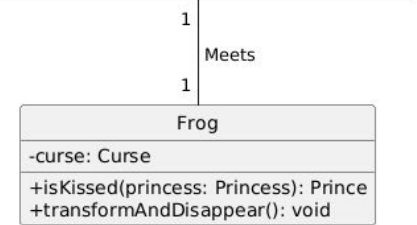
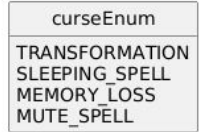
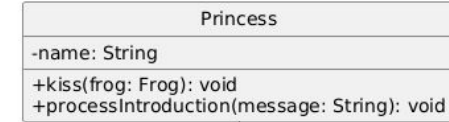


Frog Prince – DCD - Step 4



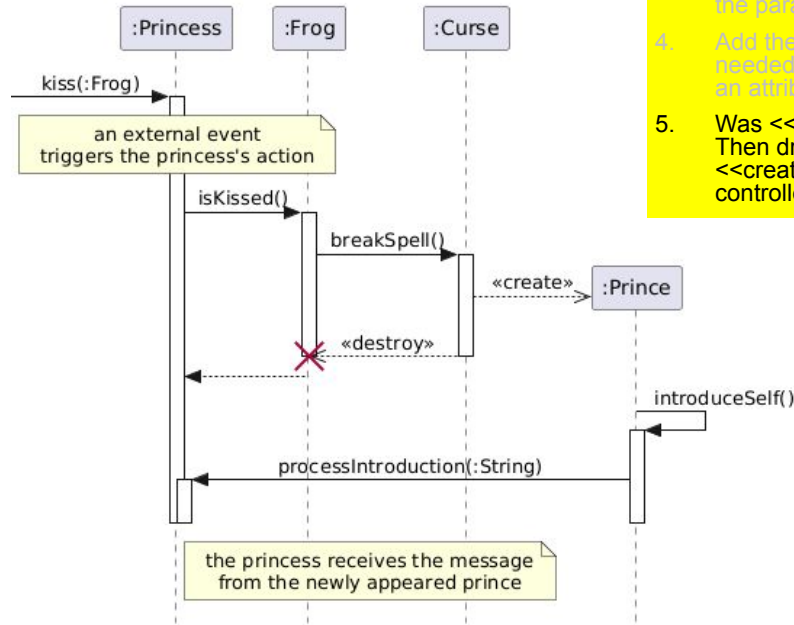
1. Start from the domain model and add the data types
2. Add the design classes (don't forget the app controller)
3. Add the operations, taking into account the parameters and return objects
4. Add the missing objects (which are needed to perform the operations) as an attribute of the calling class

DCD - The Frog Prince



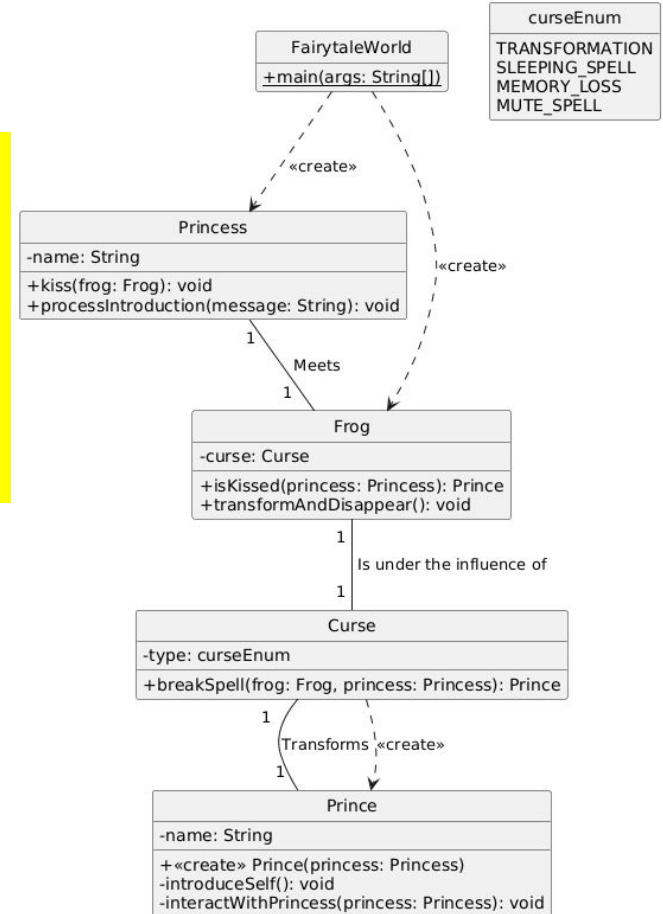
Frog Prince – DCD - Step 5

Sequence Diagram - The Frog Prince



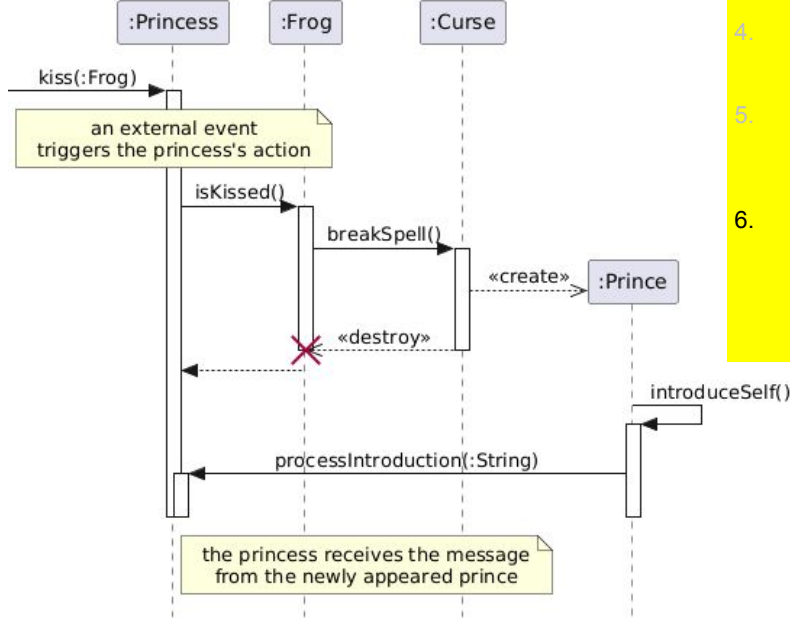
1. Start from the domain model and add the data types
2. Add the design classes (don't forget the app controller)
3. Add the operations, taking into account the parameters and return objects
4. Add the missing objects (which are needed to perform the operations) as an attribute of the calling class
5. Was `<<create>>` executed somewhere? Then draw the association with `<<create>>` (don't forget the app controller)

DCD - The Frog Prince



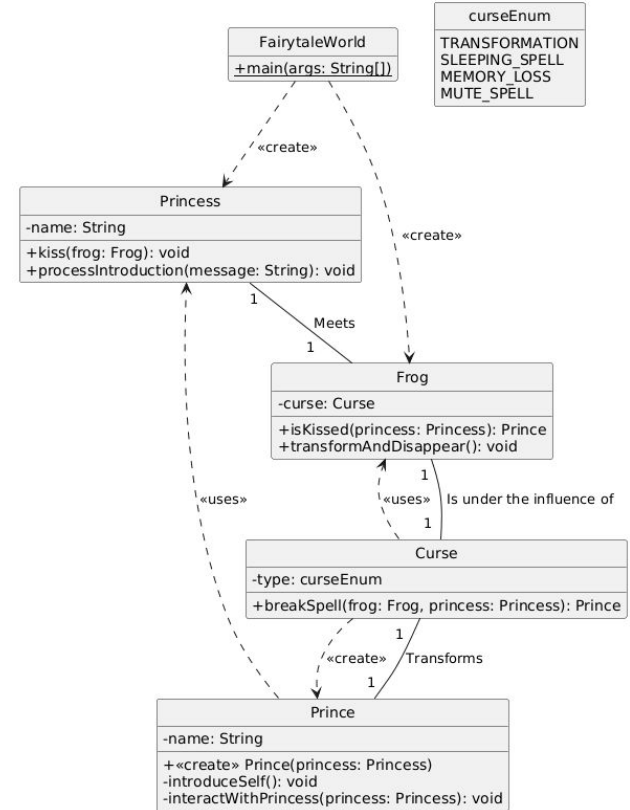
Frog Prince – DCD - Step 6

Sequence Diagram - The Frog Prince



1. Start from the domain model and add the data types
2. Add the design classes (don't forget the app controller)
3. Add the operations, taking into account the parameters and return objects
4. Add the missing objects (which are needed to perform the operations) as an attribute of the calling class
5. Was `<<create>>` executed somewhere? Then draw the association with `<<create>>` (don't forget the app controller)
6. Finally, draw all dependencies with a dashed line and an arrow indicating their direction. Who uses whom? For now, we will draw this as `<<uses>>`.

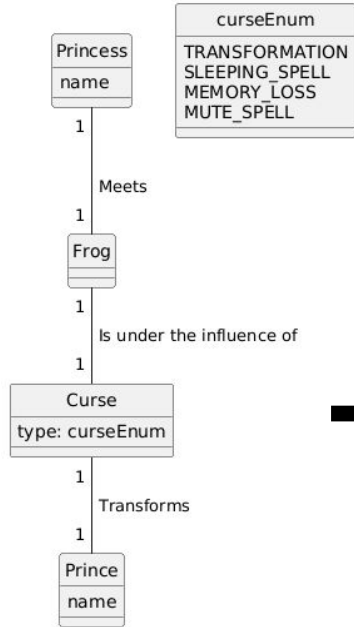
DCD - The Frog Prince



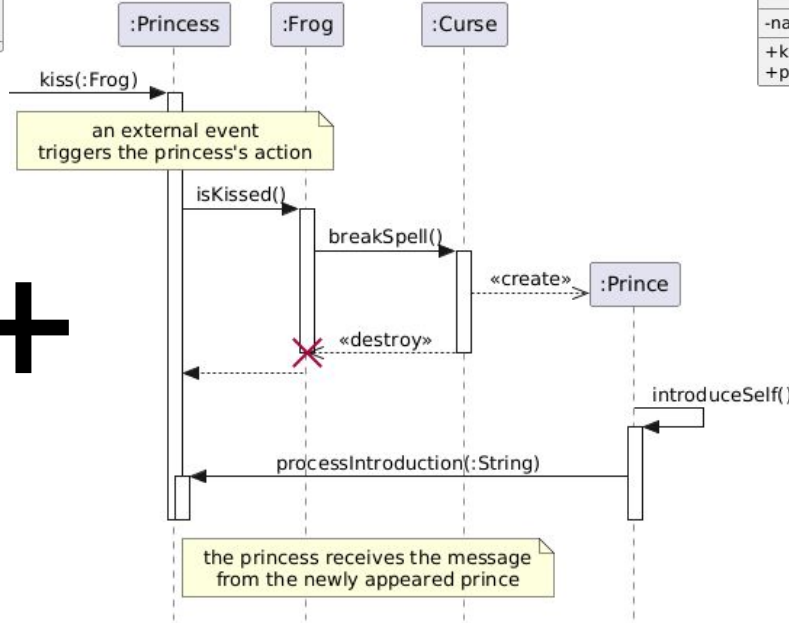
Frog Prince – DCD

The information from the domain model remains present and is expanded with the information from the SD (the FairytaleWorld is 'the' controller class)

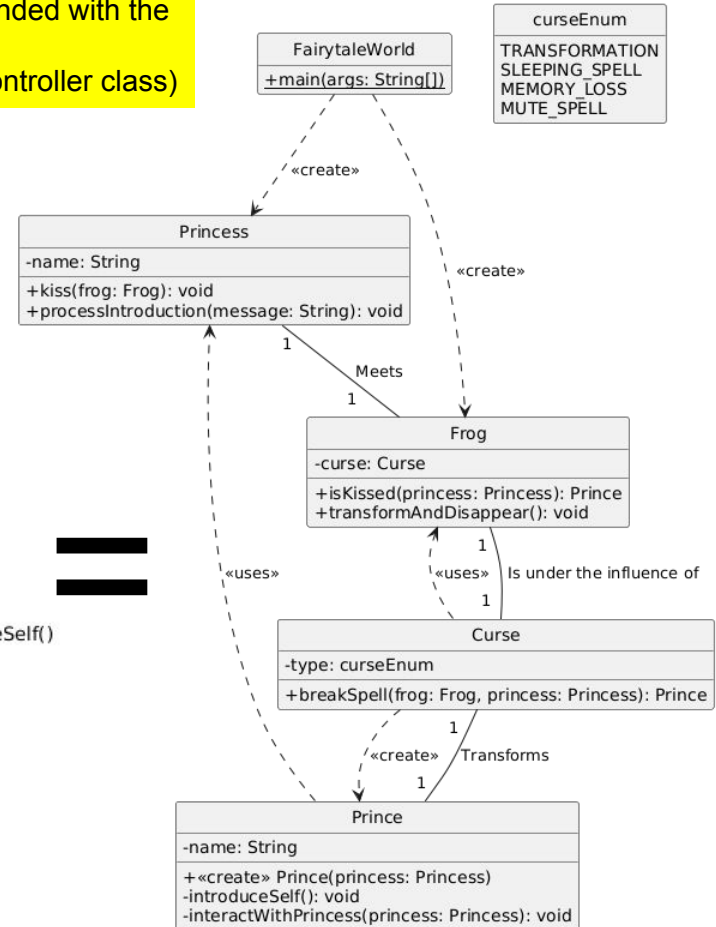
Domain model - The Frog Prince



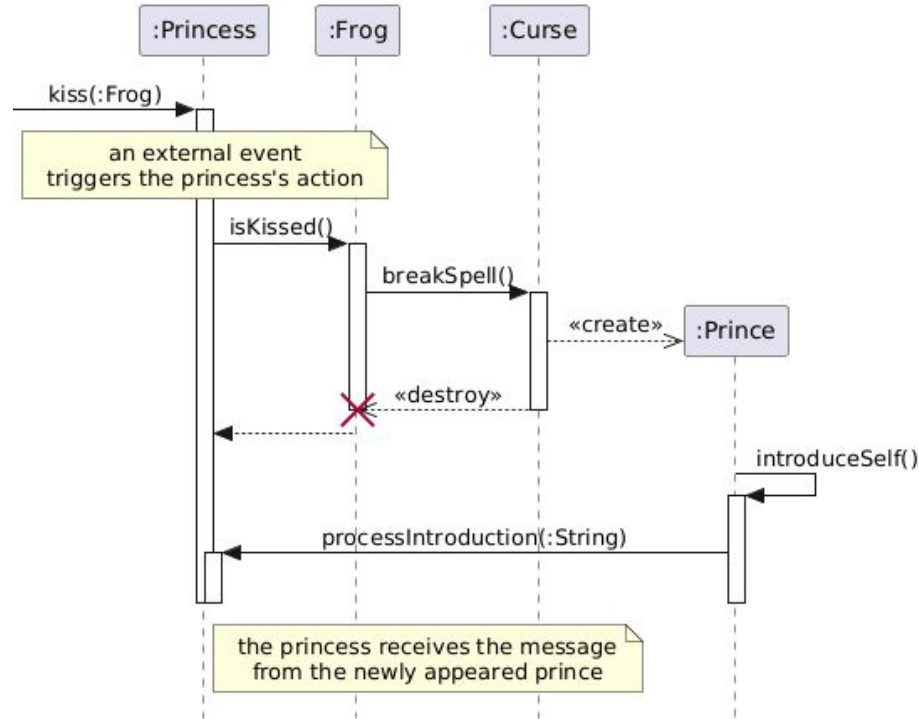
Sequence Diagram - The Frog Prince



DCD - The Frog Prince



Sequence Diagram - The Frog Prince



Java output:

The fairy tale begins...

Princess: I am going to kiss this frog.

The Frog receives a kiss.

The spell is broken...

...the essence of the frog dissolves...

...the frog transforms into a prince!

A prince appears out of nowhere!

Prince: (I introduce myself...)

Prince: (Starts a conversation with the princess)

Princess hears: "It's me, your prince!"

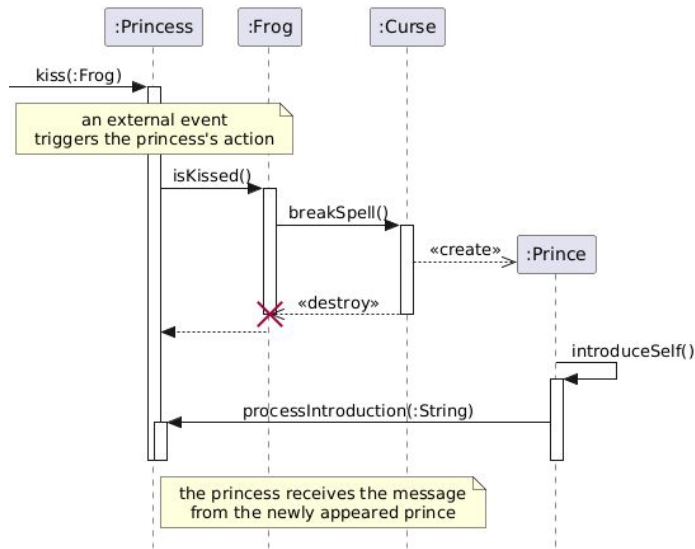
Princess: Oh, my prince!

Princess: (The whole event is now over.)

...and they lived happily ever after.

What would the Java code look like?
Use your DCD!

Sequence Diagram - The Frog Prince



```

public class Princess {
    public void kiss(Frog frog) {
        System.out.println("Princess: I am going to kiss this frog.");
        Prince prince = frog.isKissed(this);
        if (prince != null) {
            System.out.println("Princess: (The whole event is now over.)");
        }
    }
    public void processIntroduction(String message) {
        System.out.println("Princess hears: '" + message + "'");
        System.out.println("Princess: Oh, my prince!");
    }
}

```

```

public class FairytaleWorld {
    public static void main(String[] args) {
        System.out.println("The fairytale begins...");
        System.out.println("-----");
        Princess princess = new Princess();
        Frog frog = new Frog();
        princess.kiss(frog);
        System.out.println("-----");
        System.out.println("...and they lived happily ever after.");
    }
}

```

The fairy tale begins...

Princess: I am going to kiss this frog.

The Frog receives a kiss.

The spell is broken...

...the essence of the frog dissolves...

...the frog transforms into a prince!

A prince appears out of nowhere!

Prince: (I introduce myself...)

Prince: (Starts a conversation with the princess)

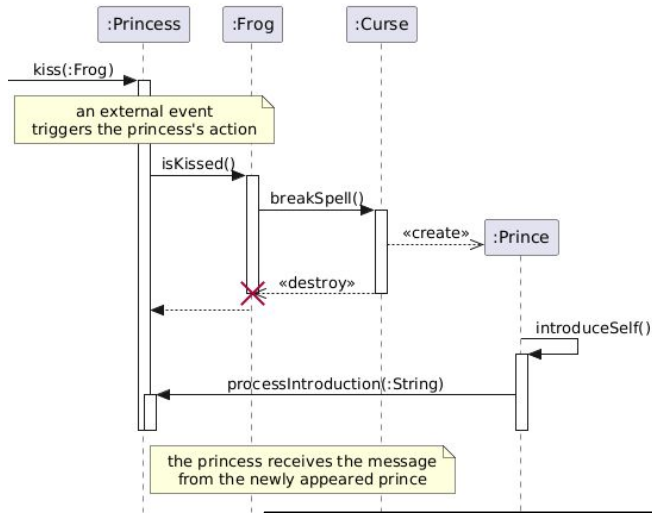
Princess hears: "It's me, your prince!"

Princess: Oh, my prince!

Princess: (The whole event is now over.)

...and they lived happily ever after.

Sequence Diagram - The Frog Prince



The fairy tale begins...

Princess: I am going to kiss this frog.

The Frog receives a kiss.

The spell is broken...

...the essence of the frog dissolves...

...the frog transforms into a prince!

A prince appears out of nowhere!

Prince: (I introduce myself...)

Prince: (Starts a conversation with the princess)

Princess hears: "It's me, your prince!"

Princess: Oh, my prince!

Princess: (The whole event is now over.)

...and they lived happily ever after.

```

public class Frog {
    private Curse curse;
    public Frog() {
        this.curse = new Curse();
    }
    public Prince isKissed(Princess princess) {
        System.out.println("The frog receives a kiss.");
        Prince result = this.curse.breakSpell(this, princess);
        return result;
    }
    public void transformAndDisappear() {
        System.out.println("...the essence of the Frog dissolves...");
        //In Java, you cannot destroy an object
    }
}
    
```

```

public class Curse {
    public Prince breakSpell(Frog frogToBeTransformed, Princess princess) {
        System.out.println("The curse is broken..");
        Prince prince = new Prince(princess);
        System.out.println("...the frog transforms into a Prince!");
        frogToBeTransformed.transformAndDisappear();
        return prince;
    }
}
    
```

```

public class Prince {
    public Prince(Princess princess) {
        System.out.println("A Prince appears out of nowhere!");
        this.introduceYourself(princess);
    }
    private void introduceYourself(Princess princess) {
        System.out.println("Prince: (I introduce myself...)");
        this.interactWithPrincess(princess);
    }
    private void interactWithPrincess(Princess princess) {
        System.out.println("Prince: (Starts a conversation with the princess)");
        princess.processIntroduction("It's me, your prince!");
    }
}
    
```



```

public class FairytaleWorld {
    public static void main(String[] args) {
        System.out.println("The fairytale begins...");
        System.out.println("-----");
        Princess princess = new Princess();
        Frog frog = new Frog();
        princess.kiss(frog);
        System.out.println("-----");
        System.out.println("...and they lived happily ever after.");
    }
}

```

```

public class Princess {
    public void kiss(Frog frog) {
        System.out.println("Princess: I am going to kiss this frog.");
        Prince prince = frog.isKissed(this);
        if (prince != null) {
            System.out.println("Princess: (The whole event is now over.)");
        }
    }
    public void processIntroduction(String message) {
        System.out.println("Princess hears: '" + message + "'");
        System.out.println("Princess: Oh, my prince!");
    }
}

```

```

public class Frog {
    private Curse curse;
    public Frog() {
        this.curse = new Curse();
    }
    public Prince isKissed(Princess princess) {
        System.out.println("The frog receives a kiss.");
        Prince result = this.curse.breakSpell(this, princess);
        return result;
    }
    public void transformAndDisappear() {
        System.out.println("...the essence of the Frog dissolves...");
        //In java, you cannot destroy an object
    }
}

```

```

public class Curse {
    public Prince breakSpell(Frog frogToBeTransformed, Princess princess) {
        System.out.println("The curse is broken...");
        Prince prince = new Prince(princess);
        System.out.println("...the frog transforms into a Prince!");
        frogToBeTransformed.transformAndDisappear();
        return prince;
    }
}

```

```

public class Prince {
    public Prince(Princess princess) {
        System.out.println("A Prince appears out of nowhere!");
        this.introduceYourself(princess);
    }
    private void introduceYourself(Princess princess) {
        System.out.println("Prince: (I introduce myself...)");
        this.interactWithPrincess(princess);
    }
    private void interactWithPrincess(Princess princess) {
        System.out.println("Prince: (Starts a conversation with the princess)");
        princess.processIntroduction("It's me, your prince!");
    }
}

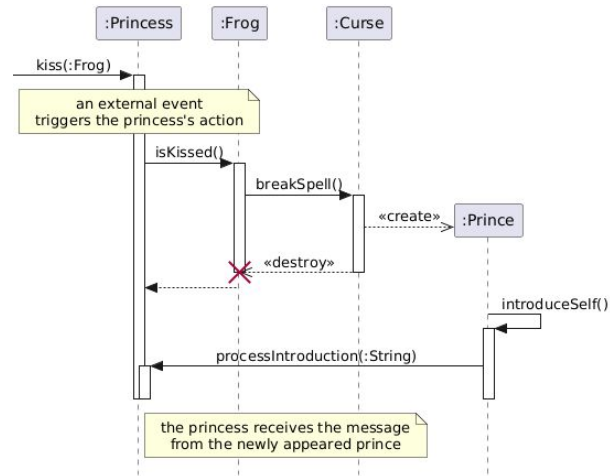
```

The fairy tale begins...

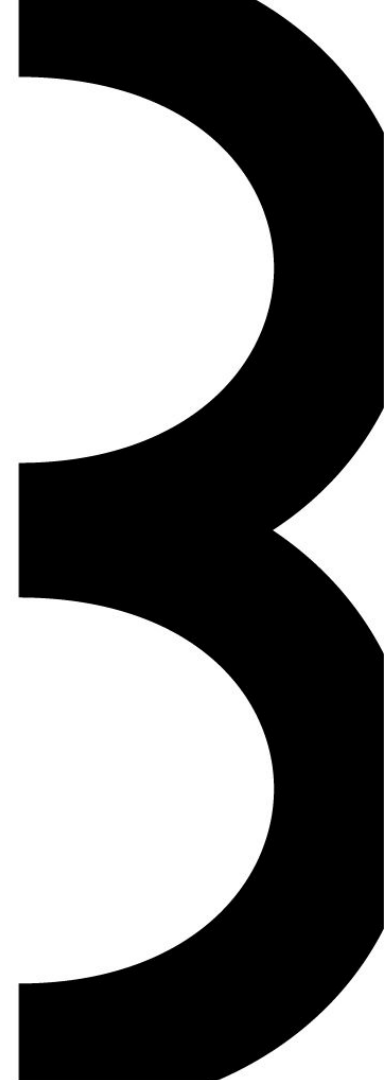
 Princess: I am going to kiss this frog.
 The Frog receives a kiss.
 The spell is broken...
 ...the essence of the frog dissolves...
 ...the frog transforms into a prince!
 A prince appears out of nowhere!
 Prince: (I introduce myself...)
 Prince: (Starts a conversation with the princess)
 Princess hears: "It's me, your prince!"
 Princess: Oh, my prince!
 Princess: (The whole event is now over.)

 ...and they lived happily ever after.

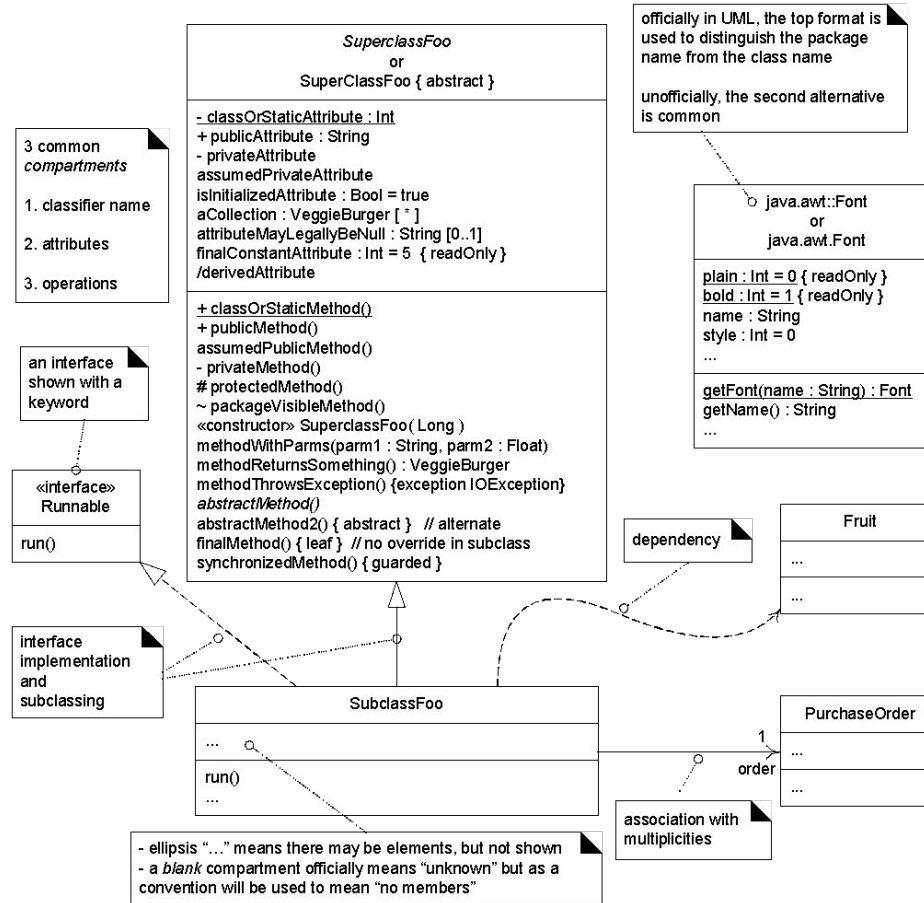
Sequence Diagram - The Frog Prince



Attributes



Notation



Accessibility attributes and operations

+ public: accessible to all classes

protected: accessible for classes within the same package and subclasses within another package

~ package private: accessible to classes within the same package

- private: accessible within the class

Convention: If you don't specify explicit accessibility

- an attribute is by default private
- an operation is by default public

Agreement in this course: always mention access!

Encapsulation

- If you modify code in a class, you must also modify all other classes that use that code:
 - a single adjustment can cause a wave of adjustments in your system
- Avoid this wave effect by encapsulation:
 - Make only a limited interface accessible to other classes
 - Hence: attribute masking (private) and controlled access via public methods (e.g. get and set methods)

Why encapsulation?

```
public class User {  
    public String birthDate; // e.g. "2025-11-17"  
}
```

```
public class ReportGenerator {  
    public void generateUserReport (User user) {  
        String dateString = user.birthDate;  
        // calculate age with String  
    }  
}
```

Suppose you decide to change the data type of an attribute
without using encapsulation:
→ Change the datatype everywhere

```
// The User class is adjusted  
public class User {  
    public LocalDate birthDate; // Changes from String to LocalDate!  
}  
+ ReportGenerator: calculate age with LocalDate
```



- public attribute
- external changes

Why encapsulation?

// The User class encapsulates its data

```
public class User {  
    private LocalDate birthDate;  
    public int getAge() {  
        return Period.between(this.birthDate, LocalDate.now()).getYears();  
    }  
}
```

```
public class ReportGenerator {  
    public void generateUserReport(User user) {  
        int age = user.getAge();  
        System.out.println("User age is: " + age);  
    }  
}
```

```
public User(LocalDate birthDate) {  
    this.birthDate = birthDate;  
}
```



GOOD

Suppose you decide to change the data type of an attribute.

Without encapsulation: → Change the data type everywhere.

With encapsulation: → The problem remains in the User class, because public getters and setters give you the ability to keep the calculations for those attributes within the same class.

In essence, the logic is encapsulated.

- private member
- advantages of the use of getters/setters...
- internal changes

Datatypes

- For each attribute, we need to define the data type
 - Integer, Double, Boolean, String/Text, DateTime...
 - Decision during design

Java

primitive datatypes

- int
- double
- boolean

objecttypes (classes)

- String
- LocalDate
- LocalTime
- LocalDateTime
- List(T)

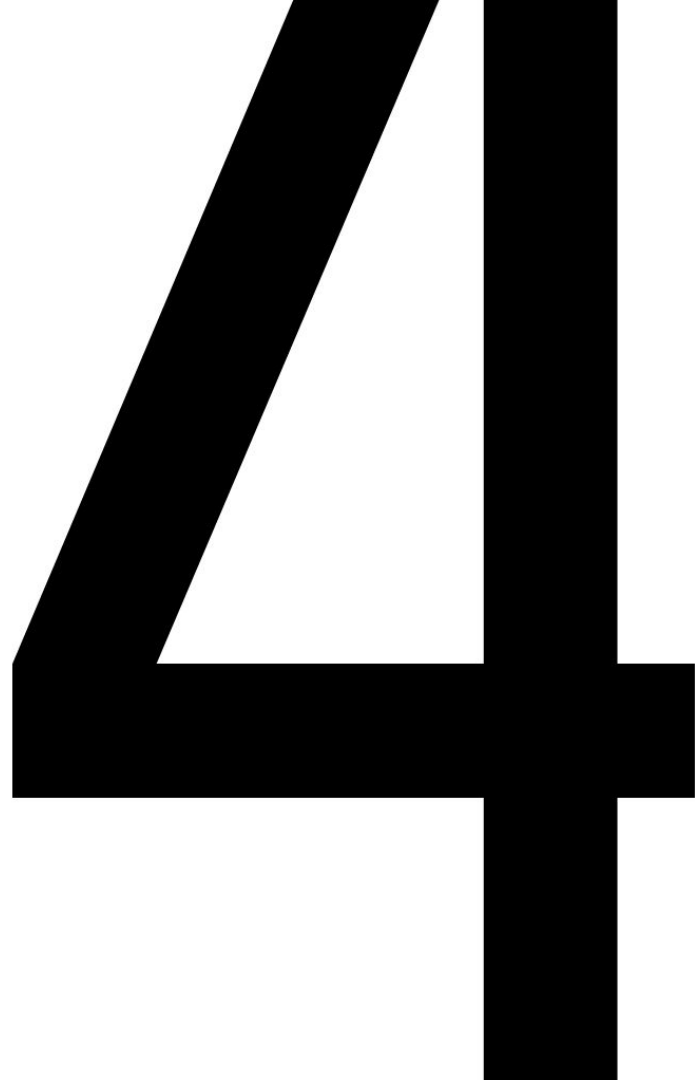
custom classes

- e.g. Register, Sale, Order...

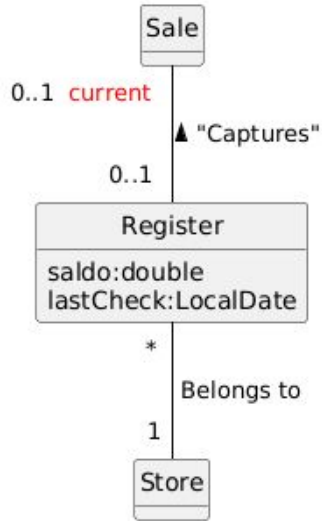
enums

...

Associations



Associations and attributes



In our analysis, we already saw that every association originates from an attribute. We discovered a hidden concept behind the attribute and wrote it as a new class to use the domain model for the database.

In the design phase, we will implement this as attributes for programming.

The "many" side always has the "one" side as an attribute.

This should also be evident in the sequence diagram.



```
public class Register {
    private double saldo;
    private LocalDate lastCheck;
    private Sale currentSale;
    private Store store;
}
```

The Java programmer combines the insights from the domain model (the concepts and their relationships) with the dynamic interactions discovered in the sequence diagrams to arrive at the final design (the Design Class Diagram).

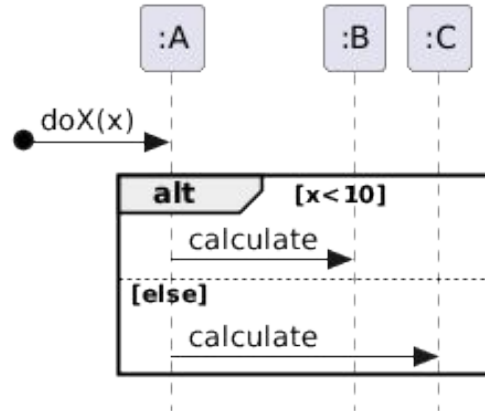
tip: don't use 'Date' in Java because it is mutable and therefore dangerous to use
`Date tomorrow = addDayTo(today);`
LocalDate is immutable (so this creates a new object when edited, like a String)
`LocalDate tomorrow = today.plusDays(1);`

Explanation: One side of the association causes an object* attribute on the many side...

Object attributes are typical design issues.

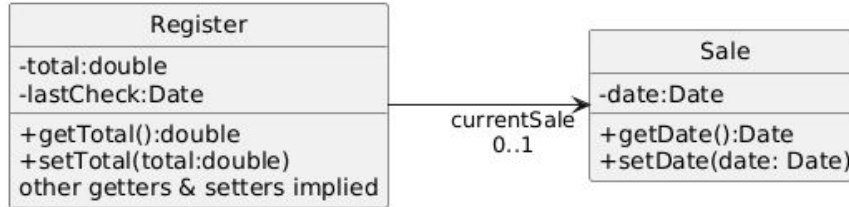
Visibility is permanent via an attribute.

Example: Exercise 8.1 Class A



```
class A {
    B b;
    C c;
    public void doX(int x) {
        if(x<10) {
            b.calculate();
        }
        else{
            c.calculate();
        }
    }
}
```

Design associations



```
public class Register {
    private double total;
    private Date lastCheck;
    private Sale currentSale;
}
```

Getters and Setters are implied in this code snippet

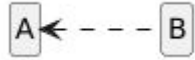
```
public class Sale {
    private Date date;
}
```

- In a DCD arrow direction indicates **visibility** of Register -> Sale.
 - Synonym: navigability
 - A reference in code points in one direction only (pointer)
- Always specify visibility in design
 - Implies that there is an attribute in the class from which the arrow departs
 - Do not mention the Sale attribute in the class (redundant information)
- Relationship name (=currentSale) always on navigable side

Arrows in the DCD



Association: B has an attribute of type A



Dependency: B uses A (for example with a method call)



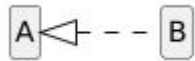
Bi-directional association: B has an attribute of type A and vice versa



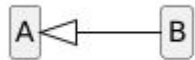
Aggregation: a relationship where the part can exist independently of the whole



Composition: a strong relationship where the part can not exist without the whole



Interface implementation: B implements A



Inheritance: B is an A

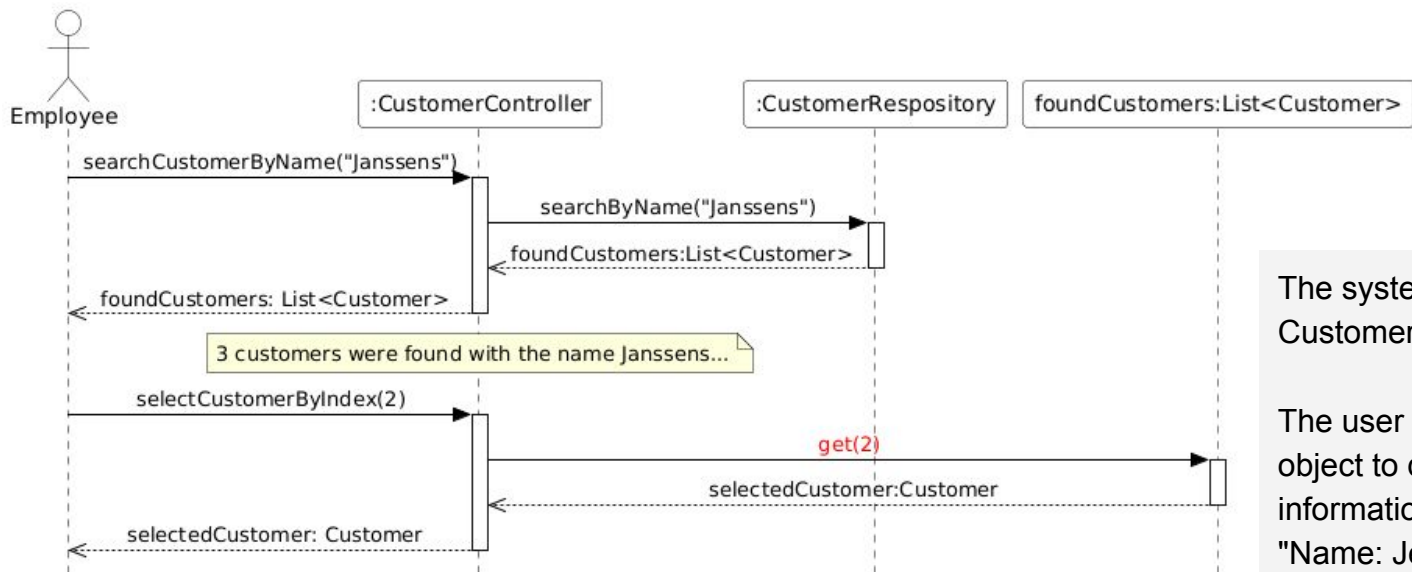
Design Classes

ClassRepository vs List<Class>

	Customer Repository or Customer Repo	List< Customer >
What?	Always all customer-objects	Subset (context specific)
Where?	Select from the database	In memory
When?	Repeatedly	(usually) Temporarily
Example	Find customer from all customers	Find customer from subset of customers

We use 'Repo' classes when we need to look up a single object from all objects of that class and we use a collection (Java: ListItem or HashMap) for temporary subsets such as the sales lines of the current sale.

ClassRepository vs List<Class>



We are not going to write the **red** operation ourselves.
It is an internal method of the List object, just like `size()`.

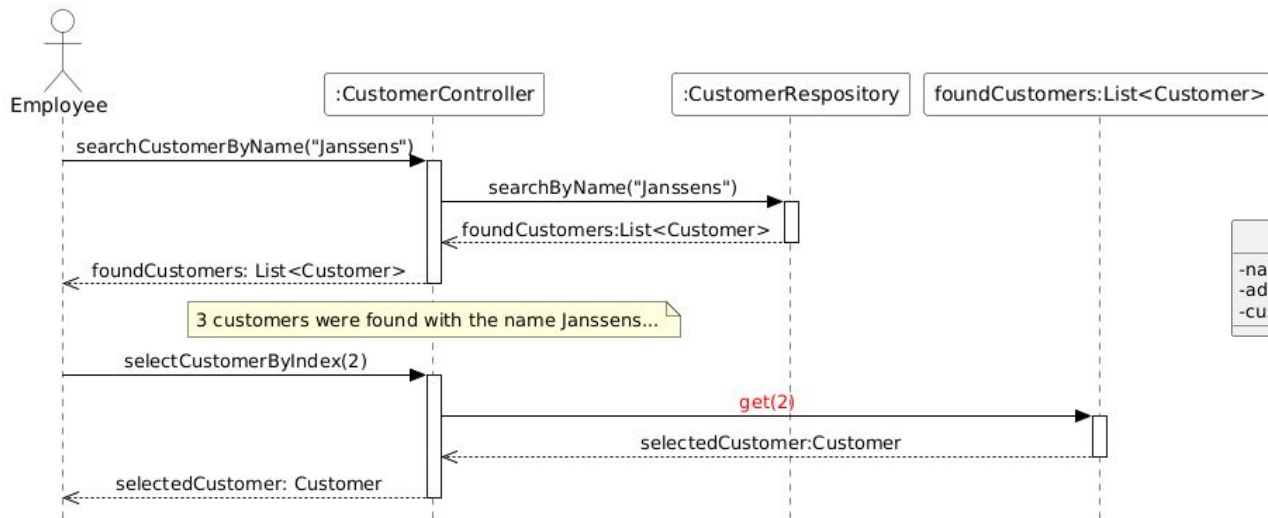
The system returns the complete Customer object.

The user interface (the screen) uses this object to display the customer's information, e.g., by filling out a form with "Name: Jef Janssens," "Address: ...," etc.

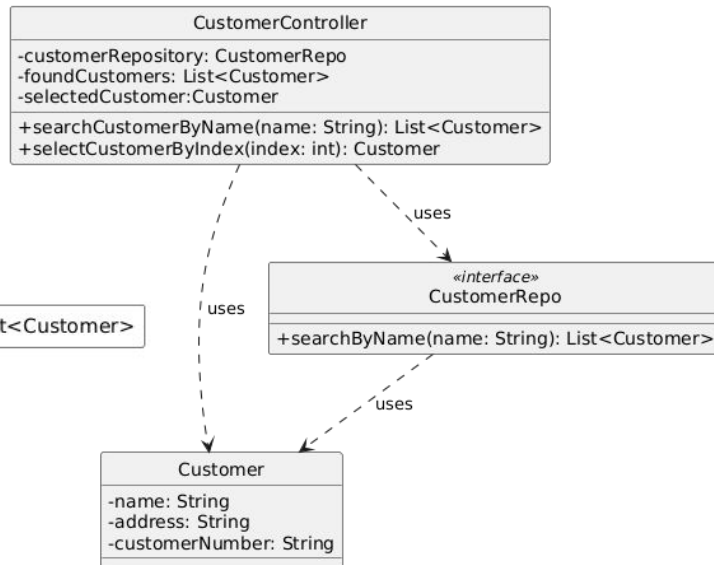
The employee can now initiate a follow-up action, such as **"makeOrderFor(selectedCustomer)"**

ClassRepository vs List<Class>

A Repo class is modeled as an interface



DCD: Search Customer (by name)



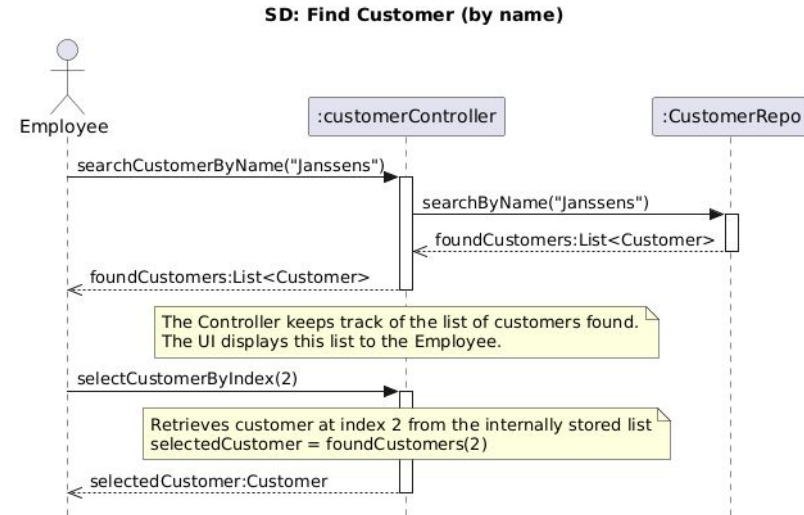
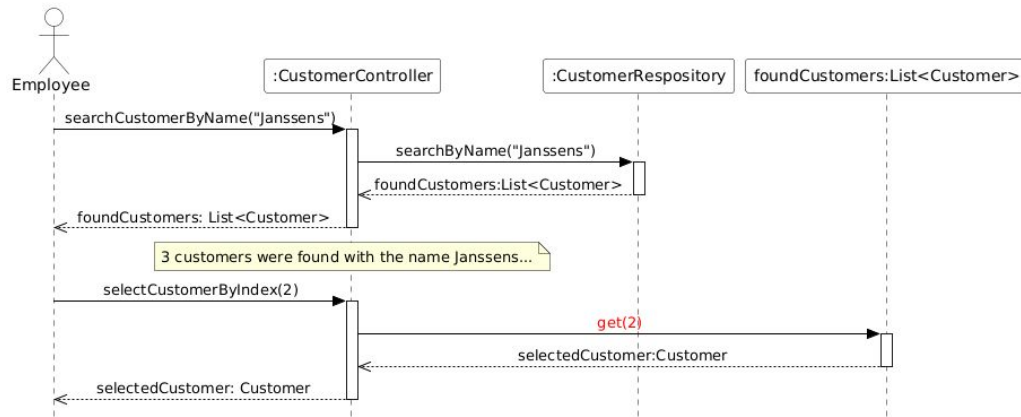
ClassRepository vs List<Class>

The List data structure is passive.

It is actually incorrect to give it a lifeline.

This is purely for educational purposes.

In reality, it will be an internally stored list by the CustomerController, as the return already indicates, and an internal search will be performed on it.



Operations



Operations vs methods

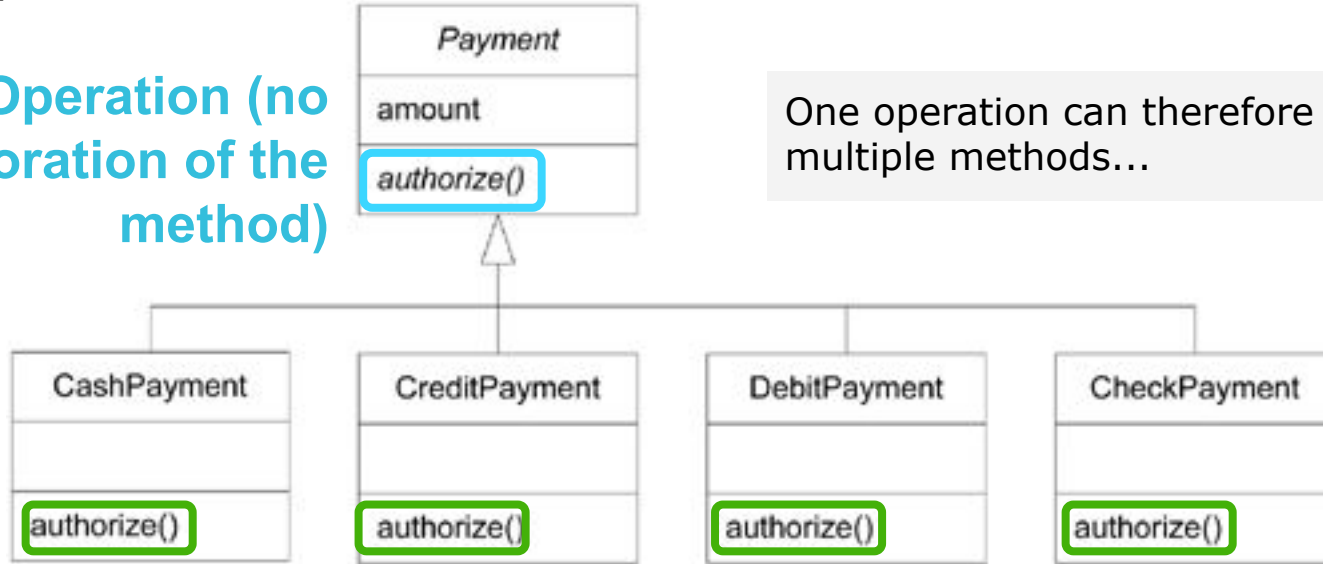
Operations	Methods
Are called on an object of a class	Are executed on an object of a class
Are basically interfaces of processes/procedures	Are the actual implementation of processes/procedures
Black-box	White-box
To be modelled	To be programmed

A method is the **implementation** of an operation.

Importance distinction Operations vs. methods

Polymorphism

Operation (no
elaboration of the
method)



One operation can therefore lead to multiple methods...

By Polymorphism, each payment type should authorize itself.
This is also in the spirit of "Do it Myself" (Coad)

However, methods
for this are being
developed

Accessibility attributes and operations

- + public
- # protected
- ~ package private
- private

```
+ classOrStaticMethod()  
+ publicMethod()  
assumedPublicMethod()  
- privateMethod()  
# protectedMethod()  
~ packageVisibleMethod()
```

Convention: If you don't specify it explicitly, accessibility for:

- an attribute is private by default
- an operation is public by default

Rule in this course: always
mention access

Syntax of operations

Form:

accessibility name (parameter list) : returntype {property text}

Components

- Accessibility: symbol for the accessibility of the operation
- Name: name of the operation in camelCase
- Parameter list: list of names and data type(s) of the input parameters
- Return type(s): the data type(s) of the return value of the operation
- Property text: additional information about the operation, for example exceptions that may occur

```
+ startMotor()  
+ getTemperature() : int  
+ setSpeed(km_per_hour : int)  
+ findUser(firstName : String, lastName: String) : User  
+ readFile(loc : String) : String {exception IOException}
```

Special operation: Constructor

- Denoted by the «create» keyword. Example:
«create» Class(:long)
- Alternative forms
 - with or without «create»:
 - Give the constructor the name of the class: SuperclassFoo(:long)
 - Name the method create: create(:long):SuperClassFoo
 - the old (UML 1) keyword «constructor»: «constructor» SuperclassFoo(:long)
- Convention: if the constructor is not mentioned, we assume there is a default constructor with the following form: + «create» ()

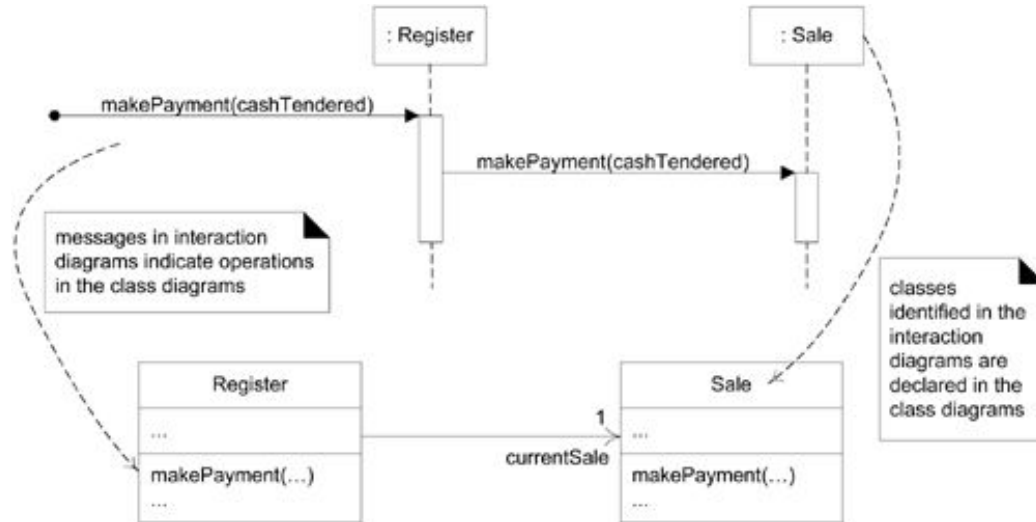
Rule in this course: we always mention the constructor in the following form: + «create» Class name (attr: attrType,...)

Special operation: Get and set operation

- By encapsulation principle (private attributes), we need to make the attributes of a class available to other classes
- Get-operation
 - To retrieve the value of an attribute
 - Get operation provided for each private attribute
 - Eg: getName(), getDOB(),... for Person class
- Set-operation
 - To adjust the value of an attribute
 - Set operation provided for each private attribute
 - Eg: setName(), setDOB(),... for Person class

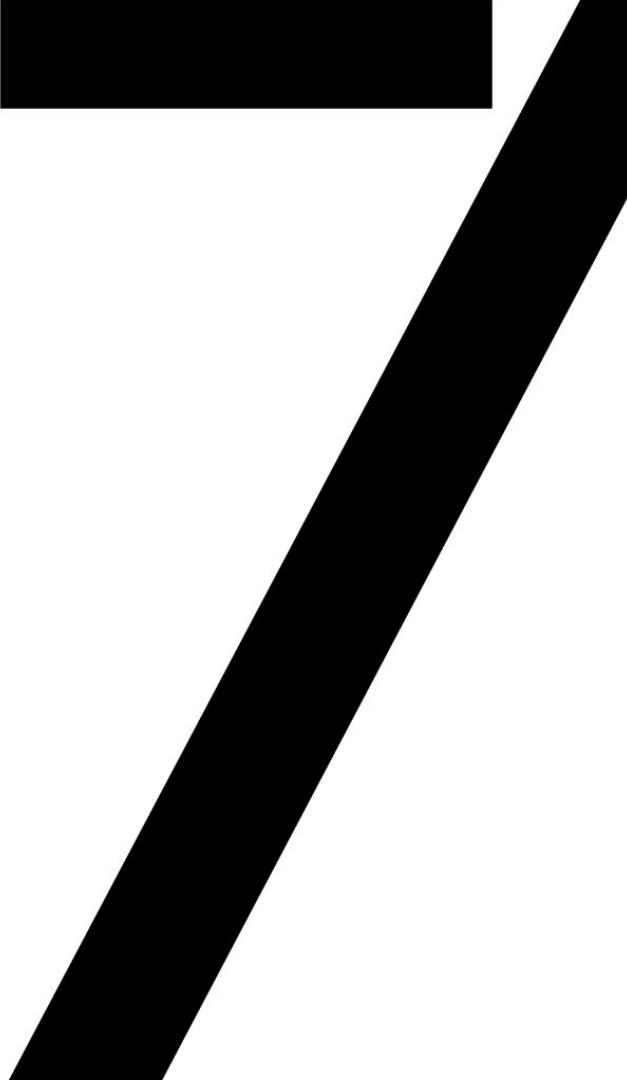
Rule in this course: we always note the minimum necessary get and set operations to show which ones we need!

From Sequence Diagram (SD) to Design Class Diagram (DCD)



- During the creation of a dynamic SD, classes and operations are defined. Attributes are listed as inputs and outputs.
- Typically an SD is drawn up first, and from that a DCD is drawn up
 - With more agile development methods (agile), this happens simultaneously

Exercises



Exercise 1

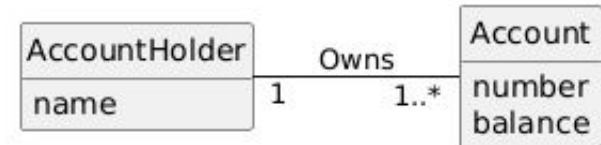
Case: Manage account

The following operations are requested:

- A method to increase the account balance (the credit or balance), i.e. to deposit an amount into the **account** = `deposit()`
- A method to **decrease** the account balance, i.e. to withdraw an amount from the account = `withdraw()`
- A method to transfer an amount to **another account** = `transfer()`
- A method that **makes the account balance available** to other objects = `getBalance()`
- A method to request all accounts of the account holder = `getAccountsOfHolder()`
- A method to request an account object using the **account number** = `getAccount()`
- **The methods** to request the **name of the account holder** from an account = `getHolder() + ?`

Provided

The domain model:



Requested

Create a DCD with the requested operations

- Think about realistic parameters
- Use the correct syntax
 - accessibility +
 - parameters +
 - data type +
 - return +
 - ...
- Use an app controller for operations that do not belong in the existing classes

Exercise 2



Case: Hotel Zeester

Use the attached domain model of Hotel Zeester.
The following operations are required.

- An operation that returns the total price of the reservation = **calculateTotalPrice()**
The total price is calculated by multiplying the agreed price per room by the number of nights.
- An operation that searches a room based on number = **getRoom()**
- An operation that searches a reservation based on number = **getReservation()**
- An operation that assigns a room to the reservation = **addRoomToReservation()**
This ultimately results in the SD in a 'found message' on Reservation, which creates a RoomReservation object to assign a room to the reservation



Provided

The domain model

Requested

Create a DCD with the requested operations

- Think about realistic parameters
- Use the correct syntax
 - accessibility +
 - parameters +
 - data type +
 - return +
 - ...
- Use an app controller for operations that do not belong in the existing classes

Recurring Example: Monopoly

Purpose of the Exercise

This exercise forms an ongoing case in the Conceptual Thinking subject, and each week we apply w digital version of the game Monopoly.

Why do we choose Monopoly?

- Everyone knows it, or almost everyone
- It contains rich structures: players, money, rules, actions, boxes, cards
- It provides an ideal context for modelling object-oriented systems
- It is tangible and recognisable, making abstract concepts concrete



Recurring Example: Monopoly

Learning objective

Students understand how theory is transformed into a working conceptual model, by:

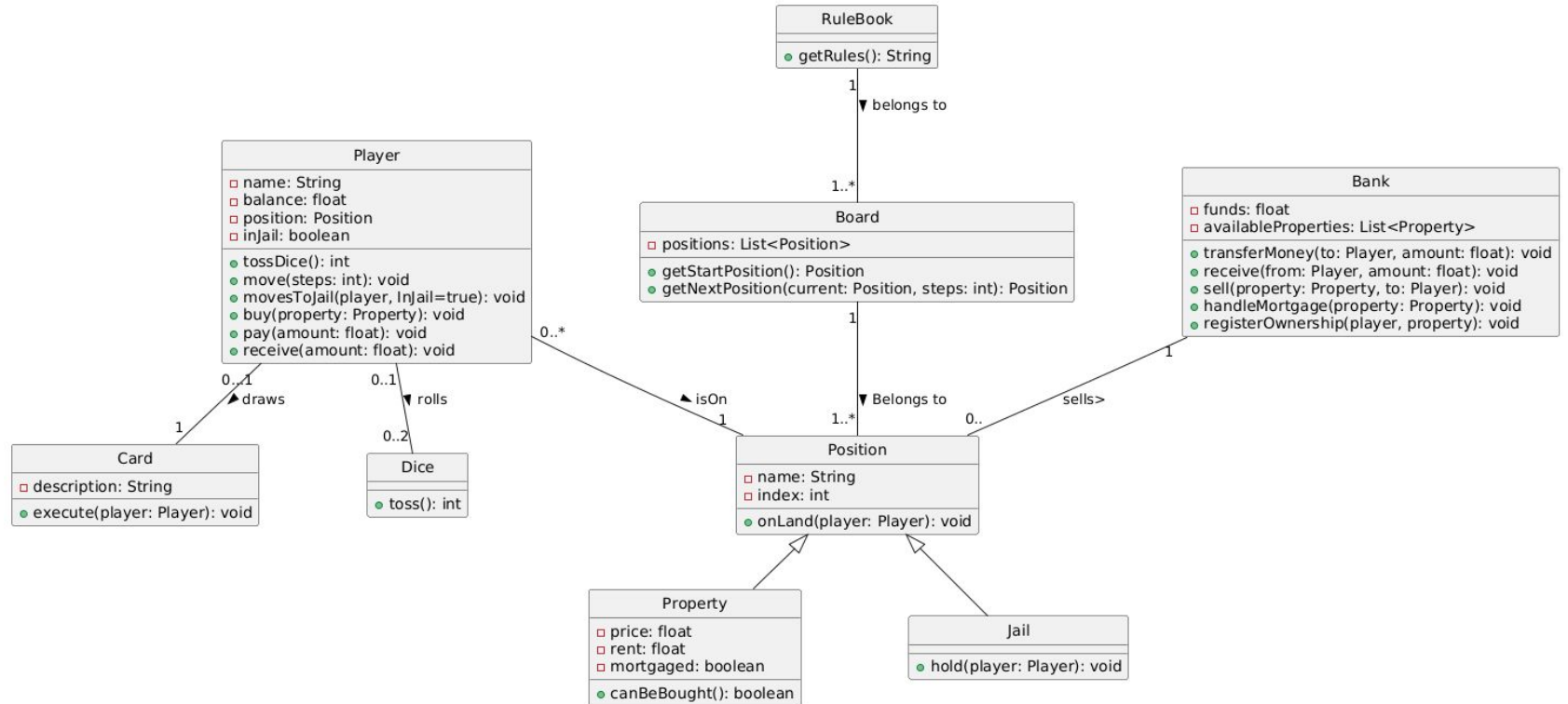
- Iterative working with one consistent case
- Apply different analysis methods (use cases, class diagrams, ...)
- Gain insight into the relationship between models and system behaviour

Short Refresh: How does Monopoly even work?

- 2-6 players move their pawn around the board with a roll of 2 dice
- Players buy properties on which they land
- Other players pay rent if they land on someone else's property
- There are special courses such as Opportunity, Community Fund, Tax, Prison, etc.
- Players can take mortgages, swap property, or go bankrupt
- Goal: become the richest player by bankrupting others

Design class diagram

Conceptual Class Diagram - Monopoly



Questions?